CAŁKOWANIE MONTE CARLO

Celem tego projektu była implementacja metod Monte Carlo do wyznaczania całek oznaczonych na przedziałach [a,b] x [c,d] z wykorzystaniem:

1) generatora wbudowanego VBA
2) Liczb Haltona

Pracę rozpoczęliśmy od implementacji algorytmu do wyznaczania całki podwójnej z użyciem wbudowanej funkcji Rnd(), która zwraca losową liczbę z przedziału [0,1].

```
Option Explicit
' Funkcja licząca całke Monte Carlo biorąca liczby z funkcji Rnd()
Public Function MC(N As Long, a As Double, b As Double, c As Double, d As Double, G As String) As Double
    Dim suma As Double
    Dim i As Long
    Dim xi As Double
    Dim yi As Double
    Dim wzor As String
    Dim war As Double

    wzor = G
    suma = 0
    ' Ustawianie innnej wartości początkowej generatora liczb rnd()
    Randomize

    For i = 1 To N
        ' Wybieranie losych liczb z przedziałów [a,b] i [c,d]
        xi = a + (b - a) * Rnd()
        yi = c + (d - c) * Rnd()

        G = Replace(G, "x", xi)
        G = Replace(G, "y", yi)
        G = Replace(G, ",", ".")

        war = Application.Evaluate(G)
        suma = suma + war

        G = wzor
    Next i

    MC = suma * (b - a) * (d - c) / N
End Function
```

Następnie utworzyliśmy funkcję robiącą to samo tylko z użyciem liczb Haltona. Do tego potrzebowaliśmy zdefiniować funkcję pomocniczą zwracającą tablicę tych liczb.

```
Private Function TabH2(N As Long) As Variant

    Dim r1 As Double
    Dim r2 As Double
    Dim f1 As Double
    Dim f2 As Double
    Dim tablica() As Double
    Dim i As Long

    ' Tworzenie tablicy rozmiaru 2 x N
    ReDim tablica(1 To 2, 1 To N)

    ' Generowanie pierwszego wiersza (Halton z bazą 2)
    For i = 1 To N
        f1 = 1
        r1 = 0
        Dim j As Long
        j = i
        Do While j > 0
            f1 = f1 / 2
            r1 = r1 + f1 * (j Mod 2)
            j = j \ 2
        Loop
        tablica(1, i) = r1
    Next i

    ' Generowanie drugiego rzędu (Halton z bazą 3)
    For i = 1 To N
        f2 = 1
        r2 = 0
        j = i
        Do While j > 0
            f2 = f2 / 3
            r2 = r2 + f2 * (j Mod 3)
            j = j \ 3
        Loop
        tablica(2, i) = r2
    Next i

    ' Zwracanie tablicy wyepłnionej liczbami Haltona jako wynik funkcji
    TabH2 = tablica
End Function
```

Dalej stworzyliśmy funkcję analogiczną co do funkcji MC, która liczy całki podwójne za pomocą liczb Haltona.

```
' Funkcja licząca podwójną całke z prostokąta [a,b] x [c,d] z funkcji g z przybliżeniem N używająca liczb Haltona
Public Function MCHalton(N As Long, a As Double, b As Double, c As Double, d As Double, G As String) As Double
    Dim suma As Double
    Dim i As Long
    Dim xi As Double
    Dim yi As Double
    Dim wzor As String
    Dim war As Double
    Dim wartosc() As Double

    ReDim wartosc(1 To 2, 1 To N)
    ' Używamy tego samego algorytmu co do pierwszej funkcji MC
    wzor = G
    suma = 0
    wartosc = TabH2(N)

    For i = 1 To N
        xi = a + (b - a) * wartosc(1, i)
        yi = c + (d - c) * wartosc(2, i)

        G = Replace(G, "x", xi)
        G = Replace(G, "y", yi)
        G = Replace(G, ",", ".")

        war = Application.Evaluate(G)
        suma = suma + war

        G = wzor
    Next i

    MCHalton = suma * (b - a) * (d - c) / N
End Function
```

Sekwencje Haltona są generowane w sposób deterministyczny co gwarantuje równomierne pokrycie przestrzeni losowania, przez co jest to bardziej efektywny sposób wybierania „losowych” punktów niż metoda wbudowana Rnd().

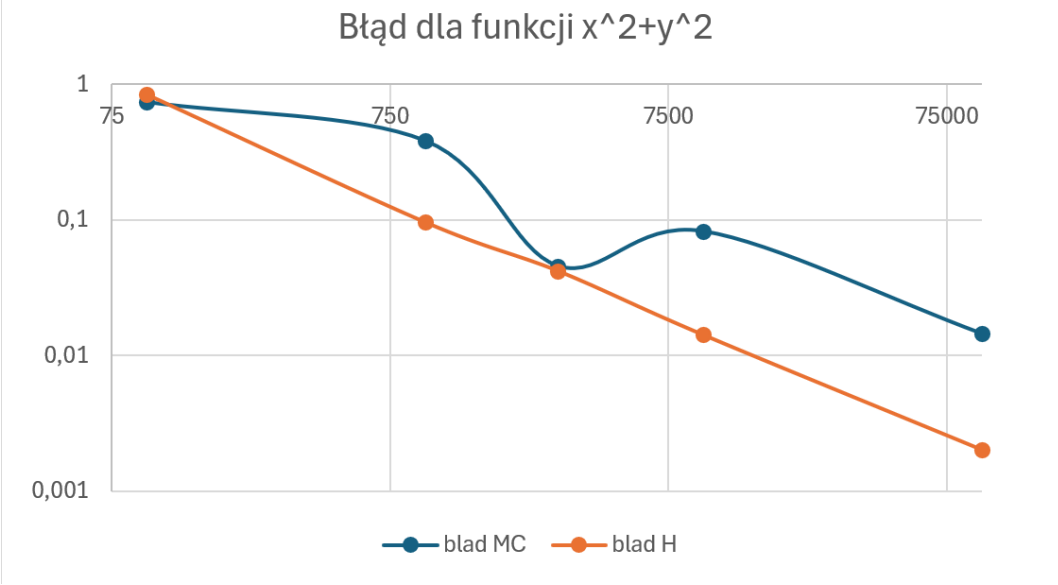
Możemy więc wysnuć hipotezę, że całki liczone właśnie za pomocą sekwencji Haltona będą dokładniejsze co do wartości niż te liczone za pomocą funkcji Rnd().

Możemy sprawdzić to porównując wyniki kilku przykładowych całek dla funkcji:

a) $F(x, y) = x^2 + y^2$ na przedziale [0, 4] x [1, 2]

Dokładna wartość tej całki to 92/3 co w przybliżeniu to 30.667

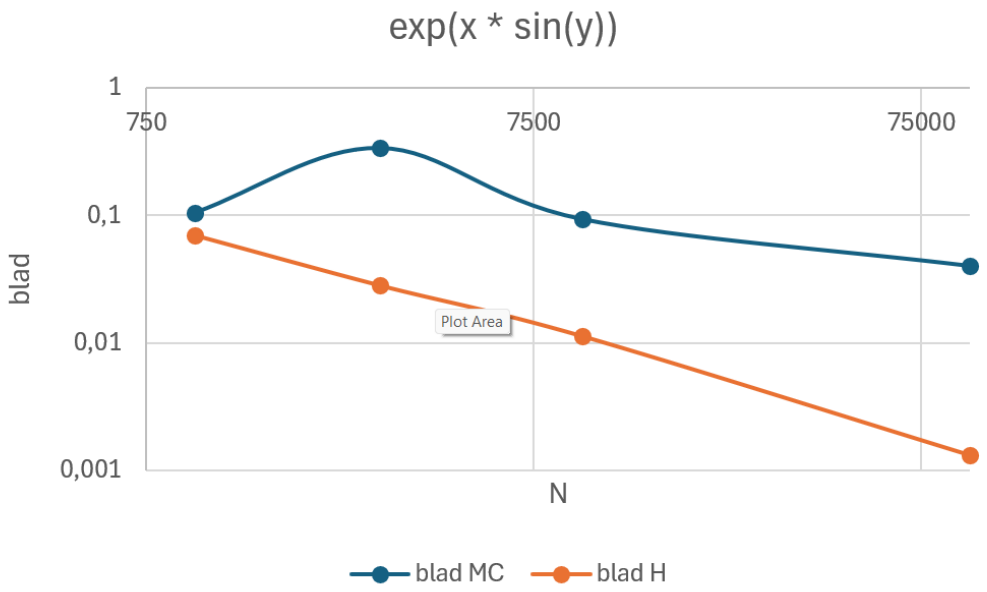
N	Metoda MC	Metoda Haltona	blad MC	blad H
100	29,93085	29,81721108	0,73615019	0,849789
1000	31,049603	30,57030988	0,38260284	0,09669
3000	30,712702	30,62486253	0,04570221	0,042137
10000	30,749261	30,65283327	0,08226146	0,014167
100000	30,681476	30,66499472	0,01447592	0,002005



b) $F(x,y) = \exp(x * \sin(y))$ na przedziale [3, 4] x [-1, 1]

Przybliżona wartość całki z tej funkcji to 7.39981

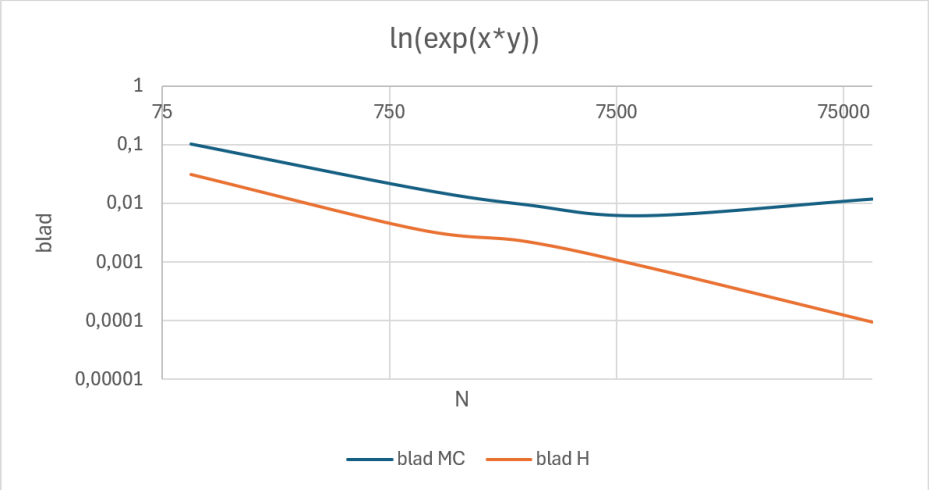
metoda MC	Metoda Haltona	blad MC	blad H
6,1388306	6,802914161	1,260979397	0,596896
7,5051894	7,329755195	0,105379414	0,070055
7,7364471	7,371490654	0,336637066	0,028319
7,4933983	7,38851378	0,093588323	0,011296
7,3594087	7,398495989	0,040401261	0,001314



c) F(x, y) = ln(exp(x * y)) na przedziale [-pi/2, 0] x [-1, 1]

Jej wartość to 0. Sprawdzimy jak radzi sobie nasza funkcja z takim zagrożeniem.

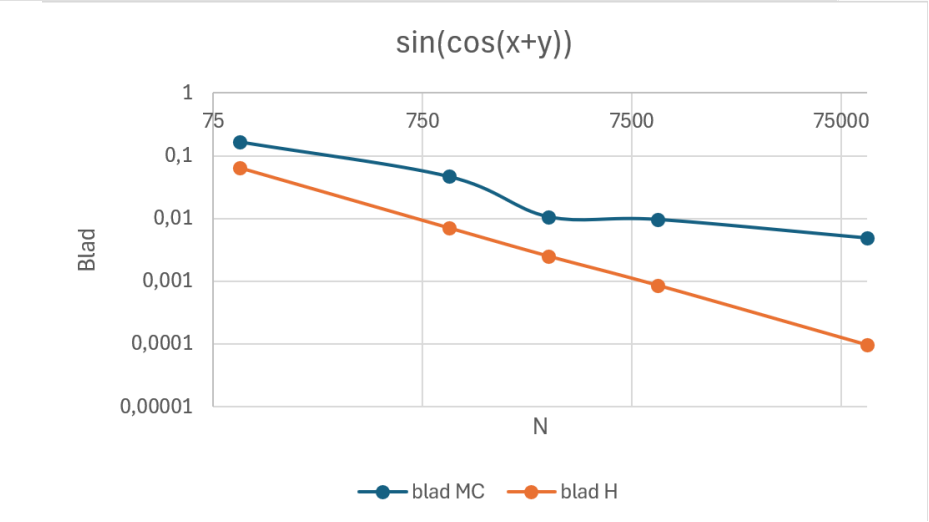
metoda MC	Metoda Haltona	blad MC	blad H
0,103856691	0,031094775	0,103857	0,031095
0,017787579	0,003621305	0,017788	0,003621
-0,009482536	0,002253201	0,009483	0,002253
-0,006172982	0,000840438	0,006173	0,00084
-0,011885138	9,63452E-05	0,011885	9,63E-05



d) F(x, y) = sin(cos(x + y)) na przedziale [-pi/2, pi/2] x [0, 1]

Jej przybliżona wartość to 1.48235

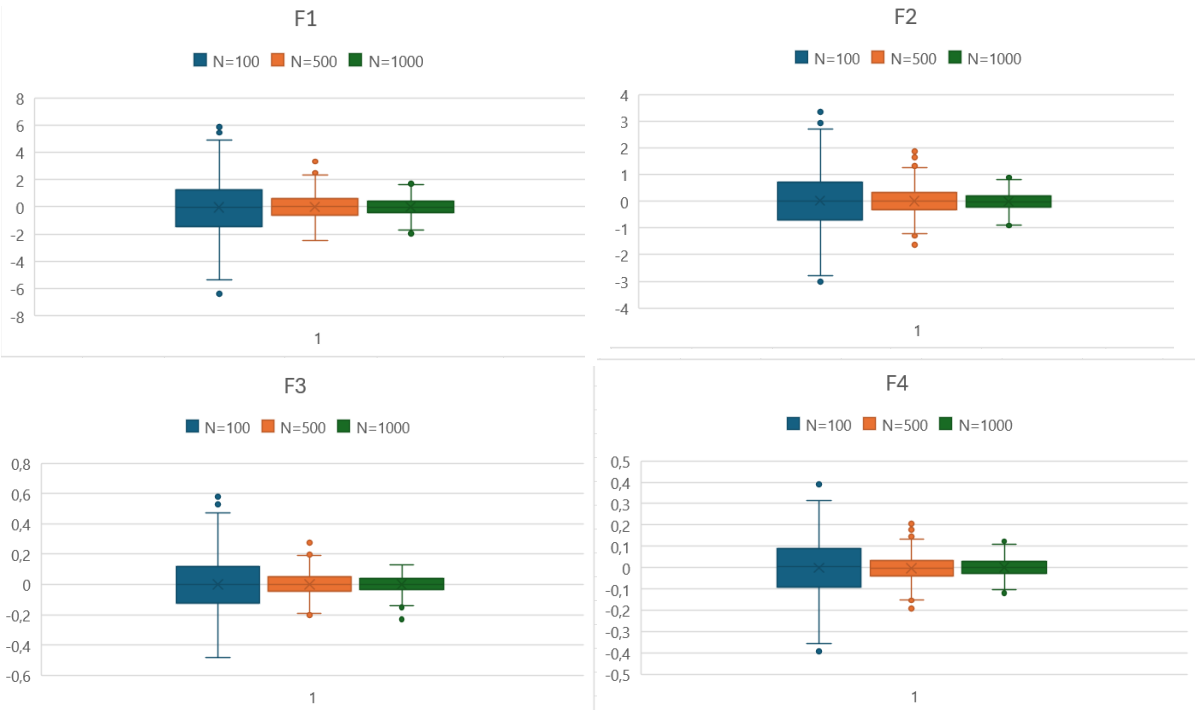
metoda MC	Metoda Haltona	blad MC	blad H
1,3166765	1,547039163	0,165674	0,064689
1,528909	1,48946432	0,046559	0,007114
1,4929856	1,484860437	0,010636	0,00251
1,4919828	1,483208843	0,009633	0,000859
1,4872334	1,482446376	0,004883	9,64E-05



Jak widać nasza hipoteza się potwierdziła. Obie metody dają akceptowalnie dobre wyniki, jednakże błąd przy metodzie Haltona jest zawsze mniejszy.

Sprawdźmy jeszcze, jak rozkładają się błędy obliczeń w zależności od N.

Poniższe wykresy „Skrzynka i wąsy” przedstawiają rozkład błędów estymatorów całek z funkcji, których używaliśmy wcześniej.



Jak widać, dla większych N-ów wartości błędów estymatorów są bardziej skoncentrowane wokół zera. Stąd łatwo wysunąć wniosek, że im większa wielkość próby, tym dokładniejszy wynik.

Warto jeszcze wspomnieć, że dla konkretnego N zestaw punktów Haltona nie zmienia się przy każdej kolejnej symulacji. Dlatego też, dla ustalonej funkcji f i ustalonego N błąd jest stały. Wynika to z deterministyczności liczb Haltona.

*W poleceniu dodatkowo było wspomniane o tym by funkcje przykładowe napisać z użyciem programowania obiektowego. Klasa znajduje się w pliku excela, jednak podczas porównywania metod wygodniej nam było korzystać ze „zwykłych” funkcji.

Projekt wykonali Karolina Gralak, Wiktor Chmiel i Marek Makowski